

# 微处理器原理与应用

---

指令系统和汇编程序设计

主讲：王鹏伟

章节：1~4章

学时：16学时

Email: [wangpw@sdu.edu.cn](mailto:wangpw@sdu.edu.cn)

# 指令系统

---

- 微处理器通过执行程序来完成指定的任务，而程序是由一系列有序指令组成，微处理器是在这些指令的控制下工作的。微处理器可以识别的每一条指令称为机器指令，每一种处理器都有自己可以识别的一整套指令，称为指令集或指令系统。不同系列的微处理器，有不同的指令系统，它是根据CPU硬件特点研制出来的，处理器执行指令时，根据不同的指令采取不同的动作，完成不同的功能，既可以改变自己内部的工作状态，也能控制其它外围电路的工作状态。

# 指令的基本内容

- 计算机的指令有**微指令**、**机器指令**和**宏指令**之分。微指令是微程序级的命令，属于硬件；宏指令是由若干机器指令组成，属于软件；机器指令介于二者之间，因而是硬件和软件的界面。
- 指令由操作码和操作数构成，8086指令的一般格式如下：

**操作码 [操作数]，[操作数]**

操作码用助记符来表示（一般是英文单词缩写）。根据指令的不同，操作数可以是一个，即单操作数，也可以是两个，即双操作数（源操作数和目标操作数），有的指令还可以没有操作数或隐含操作数。例如指令MOV AX, DX中的MOV是助记符，AX, DX为操作数（双操作数），这条指令的功能是将DX中的内容送到AX中。

# 8086系统的操作数

## (1) 立即数操作数

- 所谓立即数是指具有固定数值的操作数，即常数。它可以是字节或字（8位或16位）。存放时，该操作数跟随指令操作码一起存放在指令区，故又称为指令区操作数。

## (2) 寄存器操作数

- 操作数事先存放在某寄存器中（CPU的通用寄存器、专用寄存器或段寄存器），只要知道寄存器的名称（编号）就可以寻找到操作数。寄存器操作数既可作为源操作数，又可作为目标操作数。

## (3) 存储器操作数

- 操作数事先存放在存储器中存放数据的某个单元，只要知道存储器的地址即可寻到操作数。当然，操作数也可以存放在堆栈中（堆栈是存储器的一个特殊区域），只要知道堆栈指针，就可以用栈操作指令寻找操作数。

# 8086 CPU寻址方式

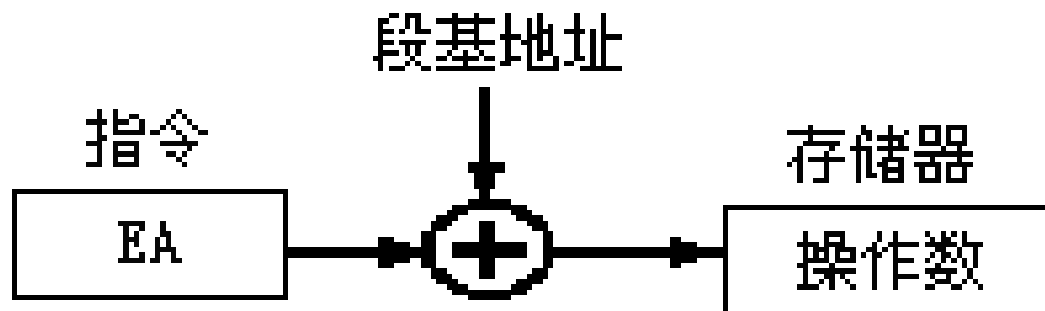
- 8086中，CS、DS、ES和SS段寄存器在程序运行过程中分别指向当前的代码段、数据段、附加段和堆栈段。而操作数可能存放在代码段中，也可能存放在数据段、附加段、堆栈段中，还可能存放在8086CPU内部的寄存器中。**存放操作数的内存单元相对于其所在段的段起始地址偏移量称为偏移地址或有效地址EA（Effective Address）。**获得操作数所在地址的方法称为寻址方式。在8086系统中，一般将寻址方式分为两类：一类是寻找操作数的地址；另一类是寻找要执行的下一条指令的地址，即程序寻址。
- |     |       |      |
|-----|-------|------|
| MOV | DST,  | SRC  |
| 助记符 | 目的操作数 | 源操作数 |

# 立即寻址方式（Immediate addressing）

- 在立即寻址（Immediate Addressing）方式下，操作数直接包含在指令中，它是一个8位或16位的常数。这类指令翻译成机器代码时，立即数作为指令的一部分紧跟在操作码之后，存放在代码段中。如果是16位数，则高8位存放在代码段的高地址单元中，低8位放在低地址单元中。**立即寻址方式常用于给寄存器赋初值，并且只能用于源操作数字段，不能用于目的操作数字段。**
- 例如：MOV AX, 3064H
- 指令执行后，（AX）= 3064H

# 直接寻址方式 (Direct addressing)

- 在这种寻址方式中，操作数存放在存储单元中，而这个存储单元的有效地址就在指令的操作码之后，操作数的物理地址可通过  $(DS) \times 16$  再加上这个有效地址形成，如下图所示。

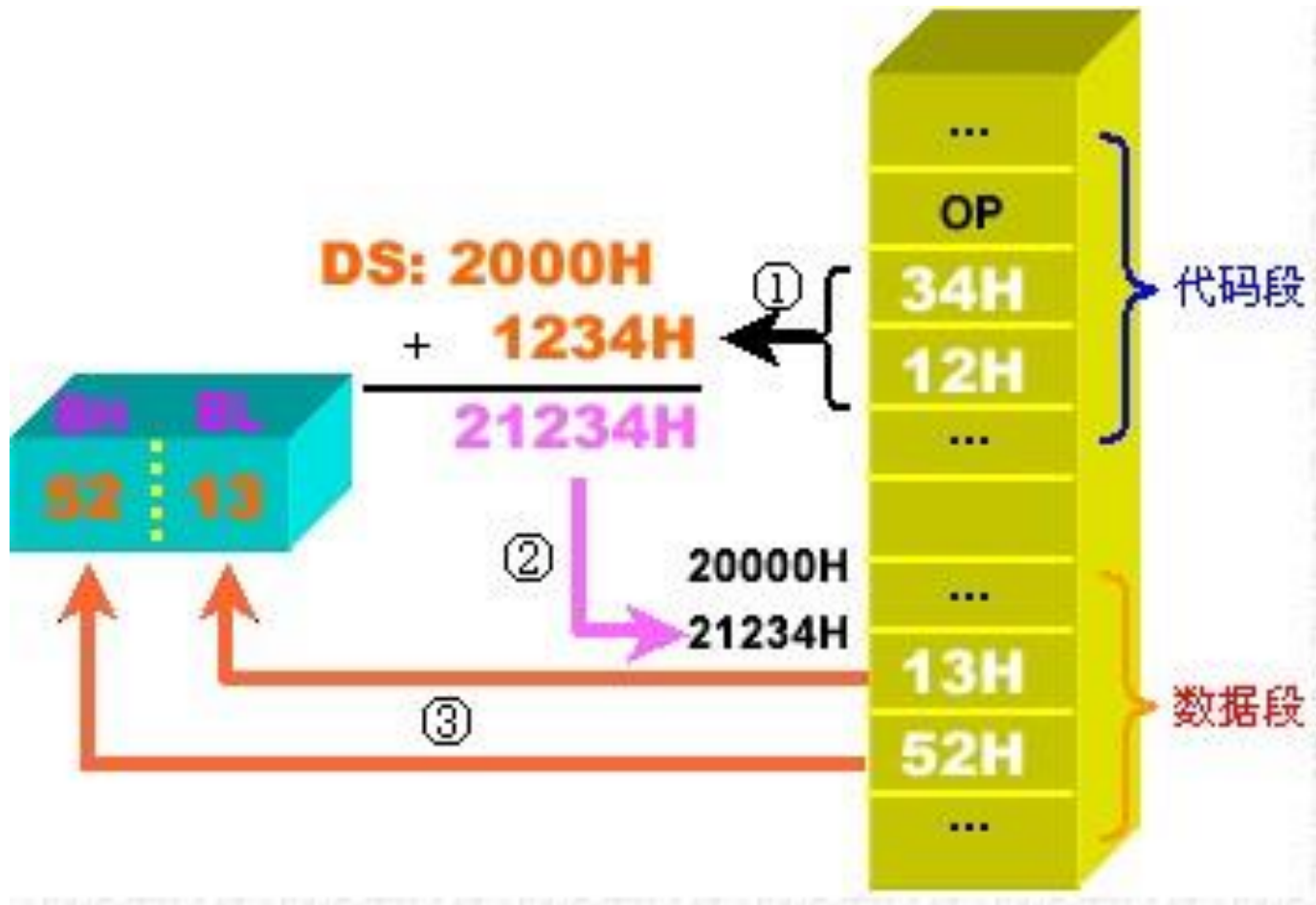


- 在汇编语言指令中，可以用符号地址（变量名或标号）代替数值地址。直接寻址方式默认操作数在数据段中，如果操作数定义在其它段中，则应在指令中指定段跨越前缀。直接寻址方式适合于处理单个变量。

# 直接寻址方式 (Direct addressing)

- 由于数据段的段寄存器默认为DS，如果要指定访问其它段内的数据，在指令中用段前缀的方式显式地书写出来。
- 带有段前缀的直接寻址方式：  
MOV ES:[1000H], AX
- 直接寻址方式常用于处理内存单元的数据，其操作数是内存变量的值，该寻址方式可在64K字节的段内进行寻址。
- 注意：立即寻址方式和直接寻址方式的书写格式的不同，直接寻址的地址要写在括号 “[ ]”内。在程序中直接地址通常用**内存变量名**来表示，如：MOV BX, VARW，其中，VARW是内存字变量。

# MOV AX, [1234H]



# 立即寻址和直接寻址

---

- 试比较下列指令中源操作数的寻址方式(VARW是内存字变量):

MOV AX, 1234H    MOV AX, [1234H]

前者是立即寻址，后者是直接寻址

MOV AX, VARW    MOV AX, [VARW]

两者是等效的，均为直接寻址

例：编写一段显示字符串**STRING**的程序

```
DATA    SEGMENT
STRING  DB    ' HAPPY NEW YEAR! ', 0DH , 0AH , ' $ '
COUNT  DW    17
DATA    ENDS
```

## (1) 直接寻址

```
    mov  dl, string
    mov  ah, 2
    int  21h                ; 显示字符 'H'

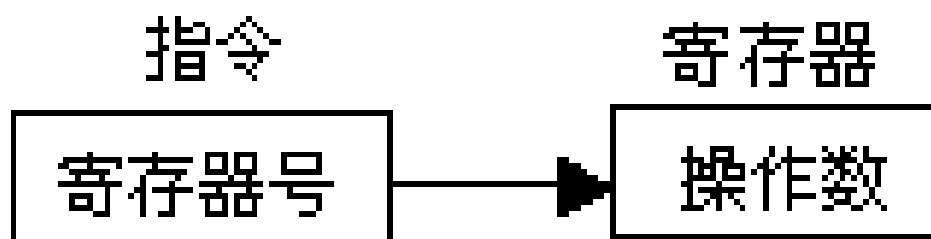
    mov  dl, string+1
    mov  ah, 2
    int  21h                ; 显示字符 'A'
    :
```

# 寄存器寻址方式（Register addressing）

- 指令所要的操作数已存储在某寄存器中，或把目标操作数存入寄存器。把在指令中指出所使用寄存器(即：寄存器的助忆符)的寻址方式称为寄存器寻址方式。
- 指令中可以引用的寄存器及其符号名称如下：
  - 8位寄存器有：AH、AL、BH、BL、CH、CL、DH和DL等
  - 16位寄存器有：AX、BX、CX、DX、SI、DI、SP、BP和段寄存器等；
  - 32位寄存器有：EAX、EBX、ECX、EDX、ESI、EDI、ESP和EBP等。
- 寄存器寻址方式是一种简单快捷的寻址方式，源和目的操作数都可以是寄存器。

# 寄存器寻址方式（Register addressing）

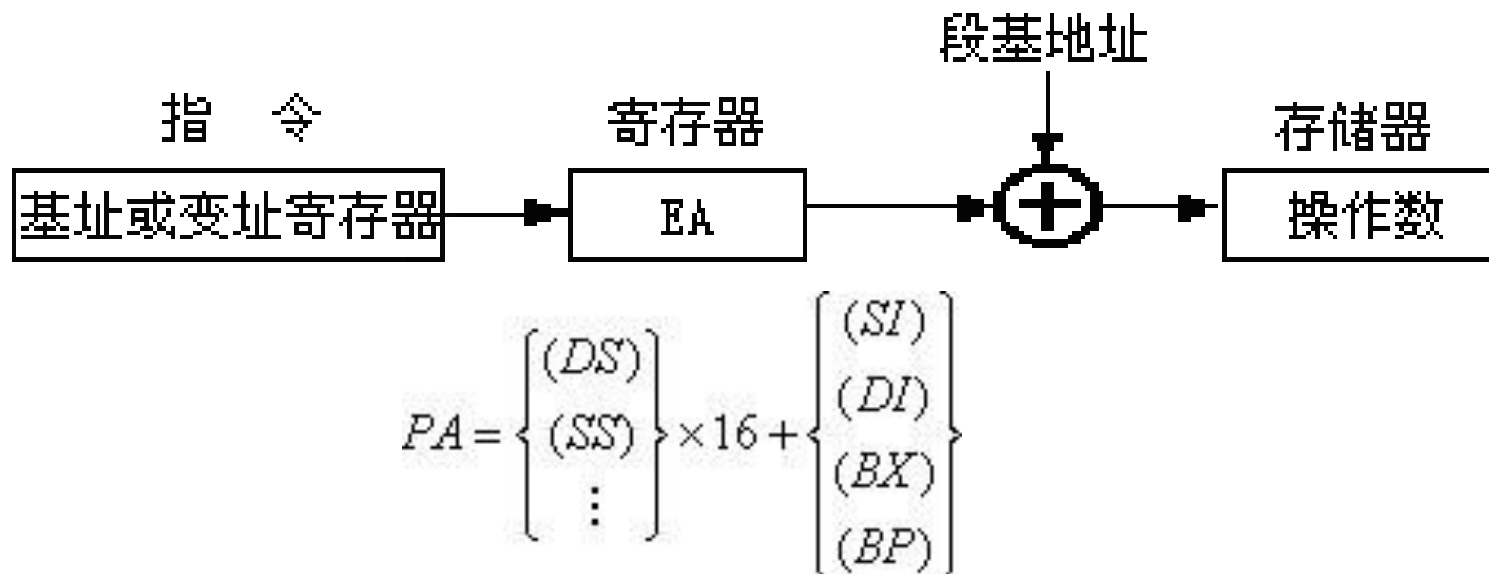
- 它使用寄存器来存放要处理的操作数，寄存器号由指令指定，如下图所示。



- 16位操作数寄存器可以是**AX、BX、CX、DX、SI、DI、SP、BP、CS、DS、ES和SS**；对于8位操作数，寄存器可以是**AL、AH、BL、BH、CL、CH、DL、DH**。由于操作数就在寄存器中，指令执行时不需要访问存储器，因此这是一种快速的寻址方式。

## 寄存器间接寻址方式（Register indirect addressing）

- 这种寻址方式通过**基址寄存器BX、BP或变址寄存器SI、DI来保存操作数的有效地址**。如果指令中使用的寄存器是SI、DI和BX，则操作数在数据段中， $((DS) \times 16)$ 再加上寄存器中的有效地址形成20位物理地址；如果指令中使用的寄存器是BP，则操作数在堆栈段中， $((SS) \times 16)$ 再加上BP中的有效地址形成20位物理地址。



**例:假设有指令: MOV BX,[DI], 在执行时, (DS)=1000H, (DI)=2345H, 存储单元12345H的内容是4354H。问执行指令后, BX的值是什么?**

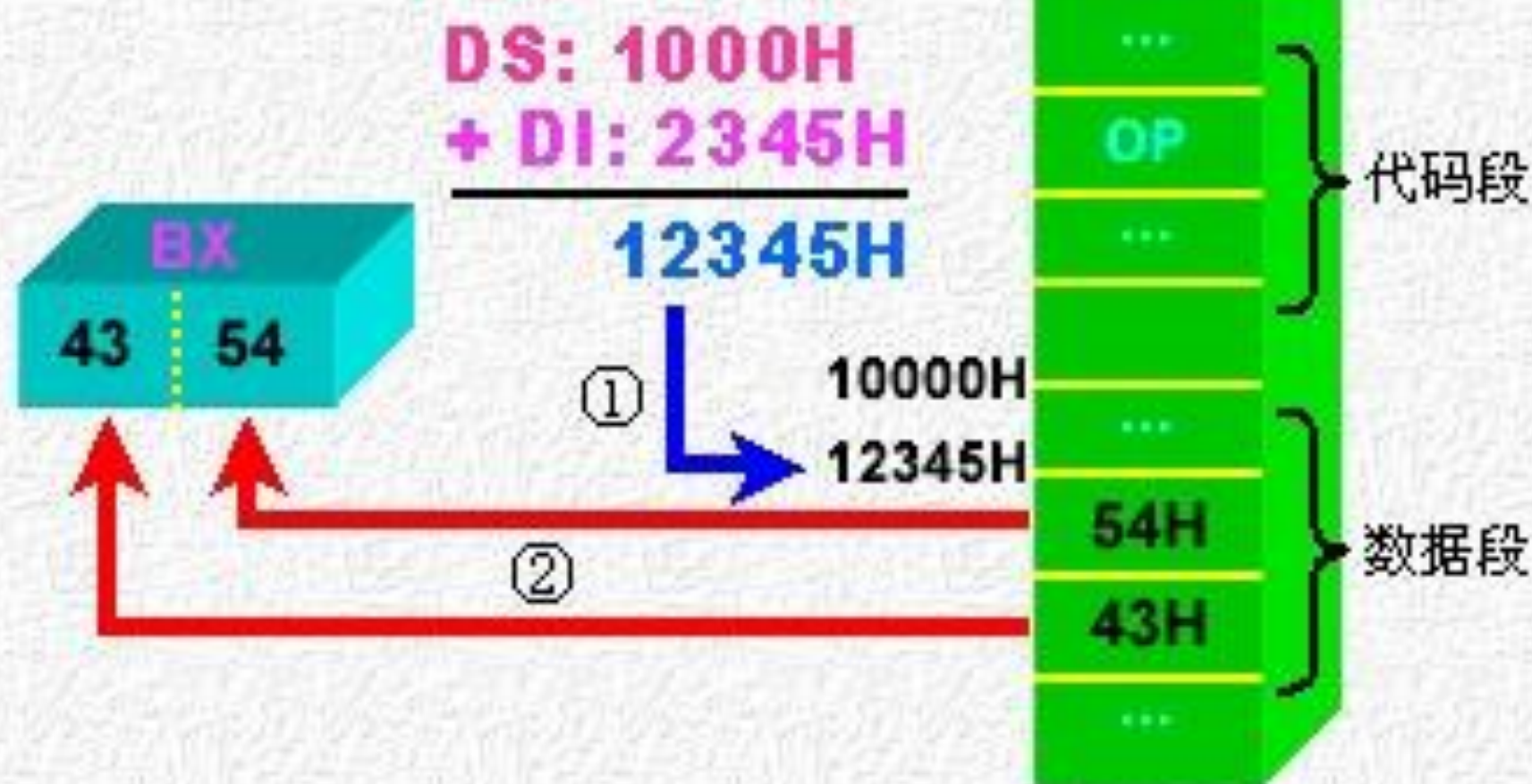
解: 根据寄存器间接寻址方式的规则, 在执行本例指令时, 寄存器DI的值不是操作数, 而是操作数的地址。该操作数的物理地址应由DS和DI的值形成, 即:

$$PA=(DS)*16+DI=1000H*16+2345H=12345H。$$

所以, 该指令的执行效果是: 把从物理地址为12345H开始的一个字的值传送给BX。

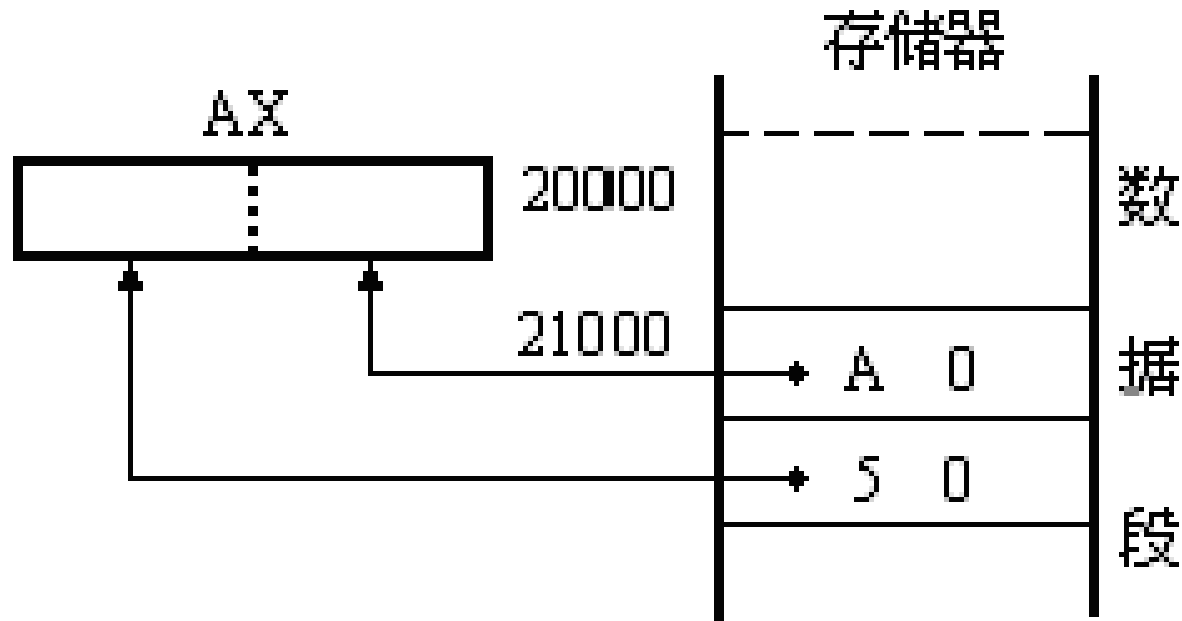
其执行过程如图所示。

$$PA = \begin{Bmatrix} (DS) \\ (SS) \\ \vdots \end{Bmatrix} \times 16 + \begin{Bmatrix} (SI) \\ (DI) \\ (BX) \\ (BP) \end{Bmatrix}$$



# MOV AX, [BX]

- 如果 (DS) = 2000H, (BX) = 1000H,  
则物理地址 = 20000H + 1000H = 21000H  
最后的执行结果为 (AX) = 50A0H。

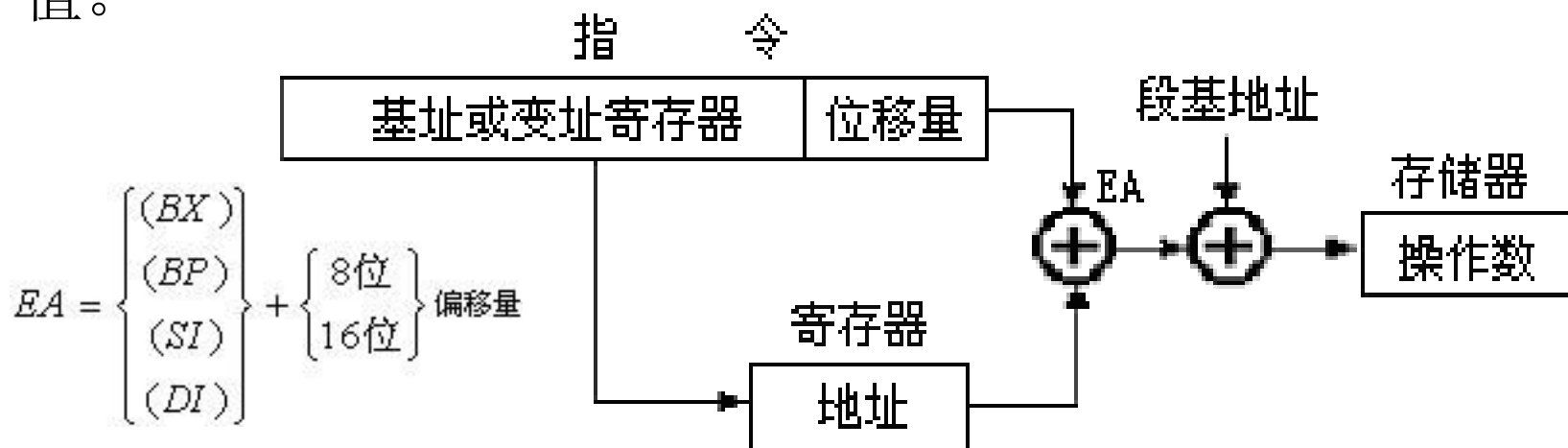


## 寄存器相对寻址方式 (Register relative addressing)

- 通过基址寄存器BX、BP或变址寄存器SI、DI与一个位移量相加形成有效地址，计算物理地址的缺省段是**SI、DI和BX为DS**，**BP为SS**。寄存器相对寻址方式也可以使用段跨越前缀。

例如：MOV AX, ES:[DI+10h]

- 寄存器相对寻址方式可用于表格处理。表格的首地址可设置为位移量，修改基址或变址寄存器的内容取得表格中的值。



**假设指令：MOV BX, [SI+100H]，在执行它时，(DS)=1000H，(SI)=2345H，内存单元12445H的内容为2715H，问该指令执行后，BX的值是什么？**

解：根据寄存器相对寻址方式的规则，在执行本例指令时，源操作数的有效地址EA为：

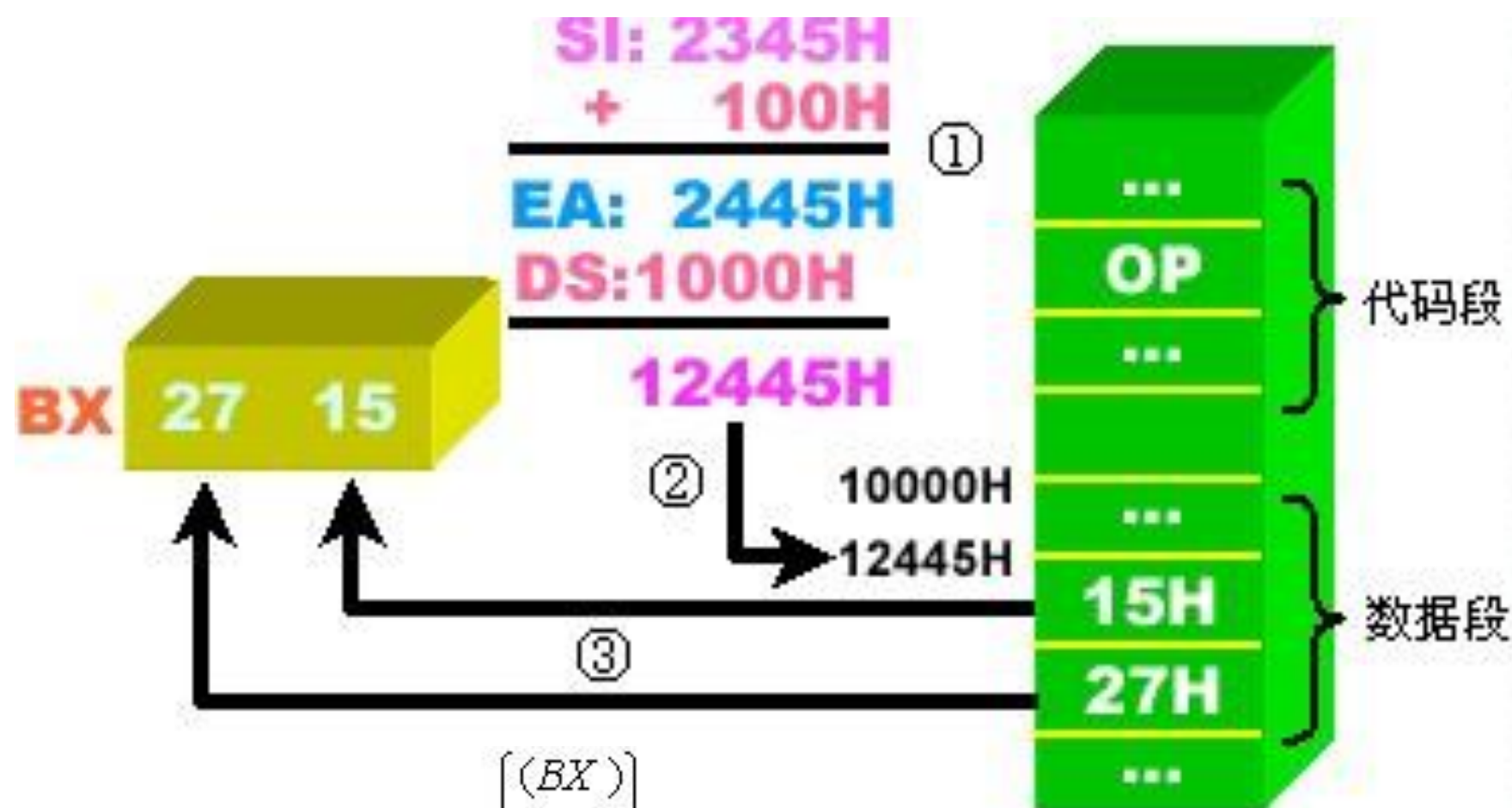
$$EA=(SI)+100H=2345H+100H=2445H$$

该操作数的物理地址应由DS和EA的值形成，即：

$$PA=(DS)*16+EA=1000H*16+2445H=12445H。$$

所以，该指令的执行效果是：把从物理地址为12445H开始的一个字的值传送给BX。

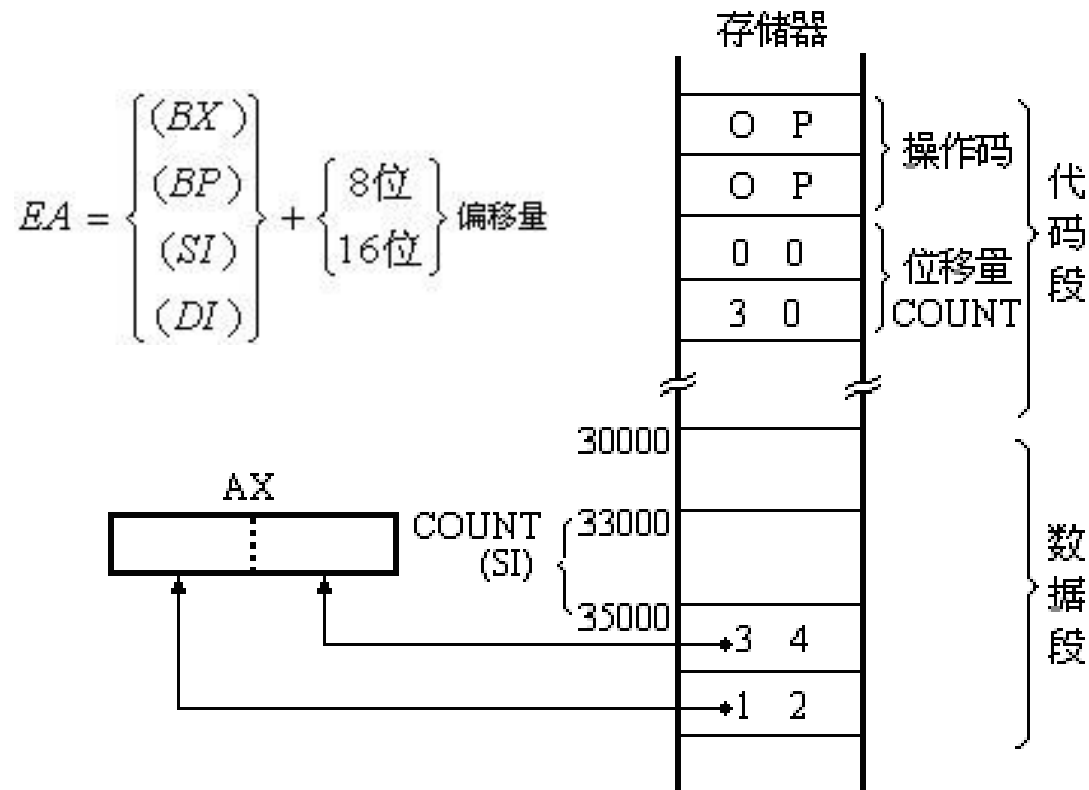
其执行过程如图所示。



$$EA = \begin{Bmatrix} (BX) \\ (BP) \\ (SI) \\ (DI) \end{Bmatrix} + \begin{Bmatrix} 8\text{位} \\ 16\text{位} \end{Bmatrix} \text{偏移量}$$

# MOV AX, COUNT[SI]

- 如果 (DS) = 3000H, (SI) = 2000H, COUNT = 3000H  
则物理地址 = 30000H + 2000H + 3000H = 35000H  
指令执行的执行结果是 (AX) = 1234H



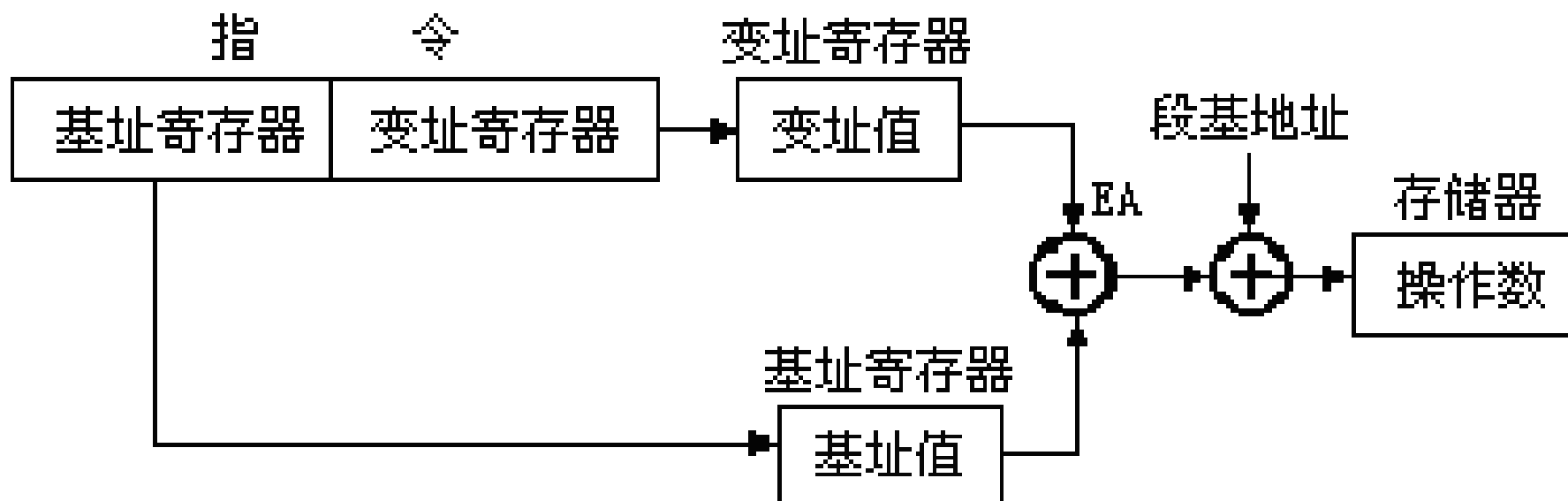
# 基址变址寻址方式 (Based indexed addressing)

- 基址加变址来定位操作数的地址，**操作数的有效地址是一个基址寄存器（BP或BX）和一个变址寄存器（SI或DI）的内容之和**。如基址寄存器为BX时，与DS形成的物理地址指向数据段；如基址寄存器为BP时，与SS形成的物理地址指向堆栈段。
- 此种寻址方式也可使用段跨越前缀。

例如：MOV AX, ES:[BX][SI]

**注意：一条指令中同时使用基址寄存器或变址寄存器是错误的。**例如：MOV CL, [BX+BP] 或 MOV AX, [SI+DI] 均为非法指令。

# 基址变址寻址方式 (Based indexed addressing)



在不使用段超越前缀的情况下，规定：如果有效地址中含有**BP**，则缺省的段寄存器为**SS**；否则，缺省的段寄存器为**DS**。

$$EA = \left\{ \begin{array}{c} (BX) \\ (BP) \end{array} \right\} + \left\{ \begin{array}{c} (SI) \\ (DI) \end{array} \right\}$$

**例:假设指令: `MOV BX, [BX+SI]`, 在执行时,  $(DS)=1000H$ ,  $(BX)=2100H$ ,  $(SI)=0011H$ , 内存单元12111H的内容为1234H。  
问该指令执行后,  $BX$ 的值是什么?**

解: 根据基址加变址寻址方式的规则, 在执行本例指令时, 源操作数的有效地址EA为:

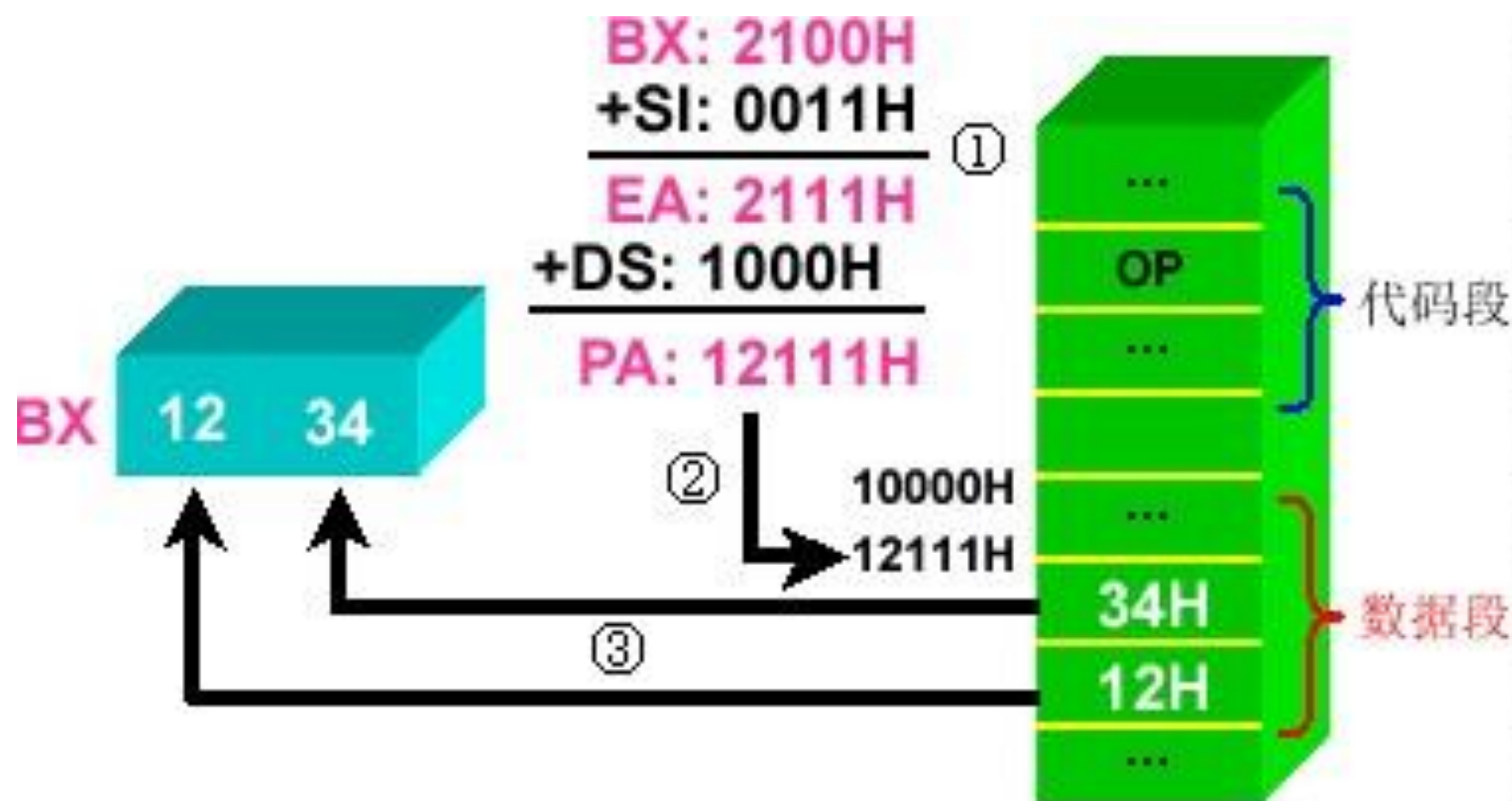
$$EA=(BX)+(SI)=2100H+0011H=2111H$$

该操作数的物理地址应由DS和EA的值形成, 即:

$$PA=(DS)*16+EA=1000H*16+2111H=12111H$$

所以, 该指令的执行效果是: 把从物理地址为12111H开始的一个字的值传送给BX。

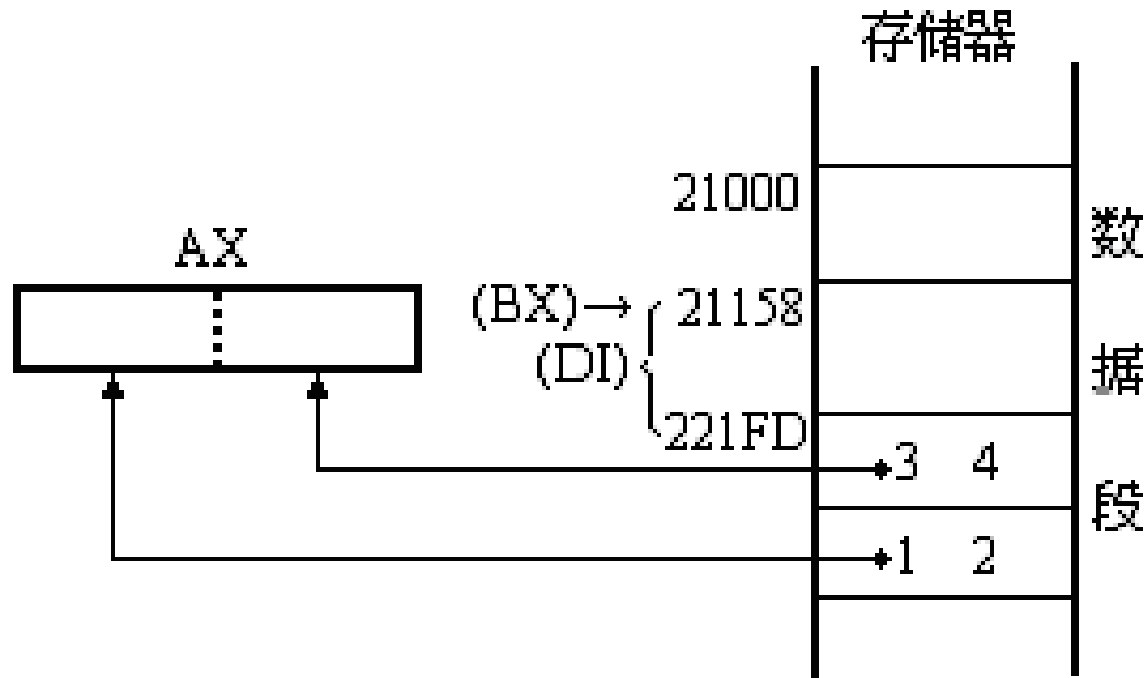
其执行过程如图所示。



$$EA = \begin{Bmatrix} (BX) \\ (BP) \end{Bmatrix} + \begin{Bmatrix} (SI) \\ (DI) \end{Bmatrix}$$

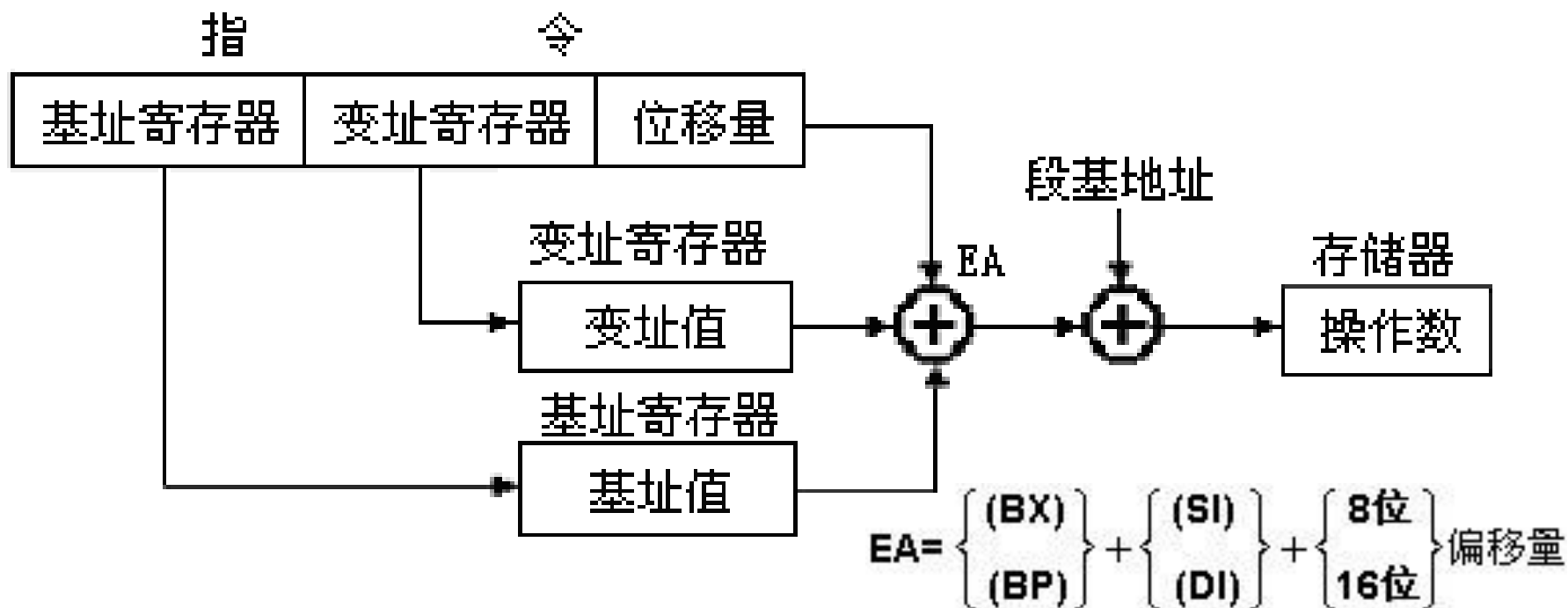
# MOV AX, [BX][DI]或 MOV AX, [BX+DI]

- $(DS) = 2100H$ ,  $(BX) = 0158H$ ,  $(DI) = 10A5H$   
则  $EA = 0158H + 10A5H = 11FDH$   
物理地址 =  $21000H + 11FDH = 221FDH$   
最后的执行结果是  $(AX) = 1234H$ 。



## 相对基址变址寻址方式 (Relative based indexed addressing)

- 这种寻址方式与基址变址寻址方式类似，不同的是基址加变址再加上一个位移量形成操作数的有效地址。缺省段的使用仍然是DS与BX组合，SS与BP组合。
- 这种寻址方式通常用于对二维数组的寻址。



# 有效地址EA

- 有效地址可以由以下三种成分组成：
  - 1) **位移量** (Displacement) 是存放在指令中的一个8位或16位数，但它不是立即数，而是一个地址。
  - 2) **基址** (Base) 是存放在基址寄存器 (BX或BP) 中的内容。它是有效地址中的基址部分，通常用来指向数据段中数组或字符串的首地址。
  - 3) **变址** (Index) 是存放在变址寄存器 (SI或DI) 中的内容。它通常用来访问数组中的某个元素或字符串中的某个字符。

有效地址的计算可用下式表示：

$$\text{EA} = \text{基址} + \text{变址} + \text{位移量}$$

**例:假设指令: `MOV AX, [BX+SI+200H]`, 在执行时, (DS)=1000H, (BX)=2100H, (SI)=0010H, 内存单元12310H的内容为1234H。问该指令执行后, AX的值是什么?**

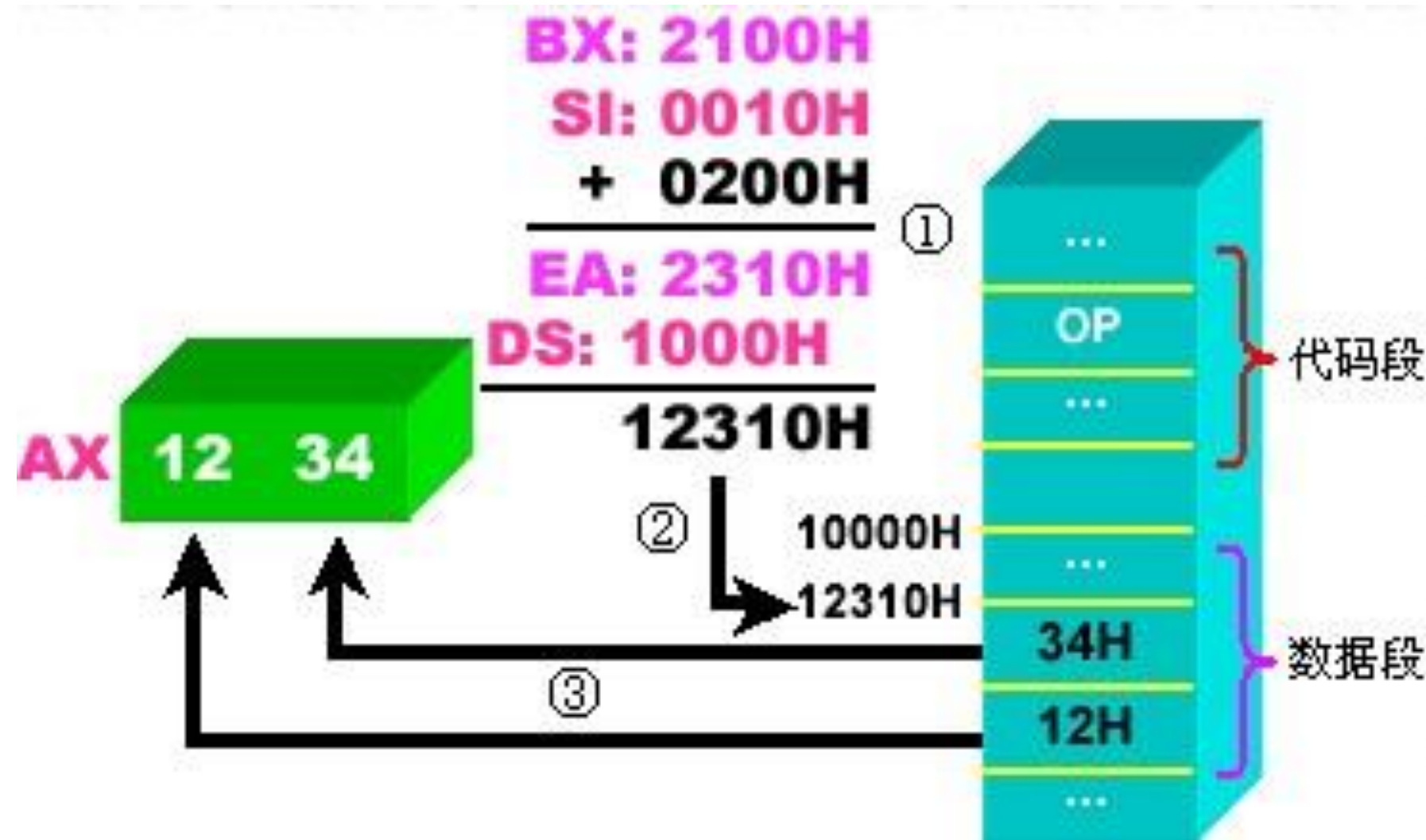
解: 根据相对基址加变址寻址方式的规则, 在执行本例指令时, 源操作数的有效地址EA为:

$$EA=(BX)+(SI)+200H=2100H+0010H+200H=2310H$$

该操作数的物理地址应由DS和EA的值形成, 即:

$$PA=(DS)*16+EA=1000H*16+2310H=12310H$$

所以, 该指令的执行效果是: 把从物理地址为12310H开始的一个字的值传送给AX。其执行过程如图所示。

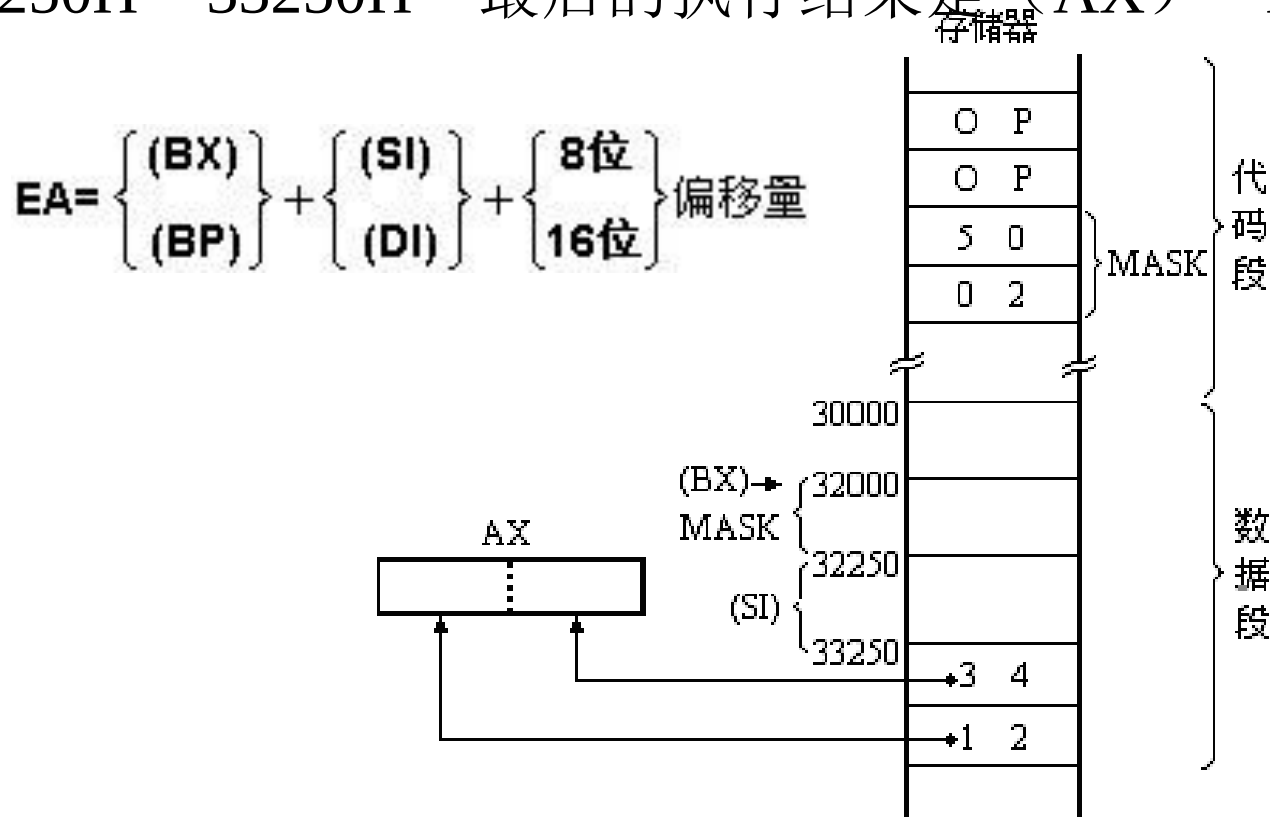


$$EA = \left\{ \begin{matrix} (BX) \\ (BP) \end{matrix} \right\} + \left\{ \begin{matrix} (SI) \\ (DI) \end{matrix} \right\} + \left\{ \begin{matrix} 8\text{位} \\ 16\text{位} \end{matrix} \right\} \text{偏移量}$$

# MOV AX, MASK[BX][SI]

- 或 MOV AX, MASK[BX+SI],

或 MOV AX, [MASK+BX+SI], MOV AX, MASK[SI][BX])  
如 (DS) = 3000H, (BX) = 2000H, (SI) = 1000H,  
MASK = 0250H, 则物理地址 = 30000H + 2000H + 1000H +  
0250H = 33250H 最后的执行结果是 (AX) = 1234H。



# 寻址例题

---

- 设BX=0123H DI=1000H DS=3200H，默认DS作为操作数对应的段寄存器(假定没使用段前缀)，试指出下列指令的寻址方式，并写出其操作数的有效地址和物理地址。
  - (1) MOV AX, [1A38H]
  - (2) MOV AX, [BX]
  - (3) MOV AX, [BX+1A38H]
  - (4) MOV AX, [BX+DI]
  - (5) MOV AX, [BX+DI+1A38H]

# 例题

---

- 8088/8086存储器分段，每个段不超过（）  
（A）64K个字                      （B）64K个字节  
（C）1兆个字节                      （D）32K个字节
- 8088/8086系统执行传送指令MOV时（）  
（A）影响CF方向标志   （B）影响DF方向标志  
（C）影响SF方向标志   （D）不影响标志位
- 表示过程定义结束的伪指令（）  
（A）END                      （B）ENDS  
（C）ENDP                      （D）ENDC

| 源操作数       | 指令的变形                                 | 源操作数的寻址方式   |
|------------|---------------------------------------|-------------|
| 只有偏移量      | MOV AX, [100H]                        | 直接寻址方式      |
| 只有一个寄存器    | MOV AX, [BX] 或 MOV AX, [SI]           | 寄存器间接寻址方式   |
| 有一个寄存器和偏移量 | MOV AX, [BX+100H] 或 MOV AX, [SI+100H] | 寄存器相对寻址方式   |
| 有二个寄存器     | MOV AX, [BX+SI]                       | 基址加变址寻址方式   |
| 有二个寄存器和偏移量 | MOV AX, [BX+SI+100H]                  | 相对基址加变址寻址方式 |

| 指令的举例                        | 访问内存单元所用的段寄存器 |
|------------------------------|---------------|
| MOV AX, [123456H]            | ；默认段寄存器DS     |
| MOV EAX, [EBX+EBP]           | ；默认段寄存器DS     |
| MOV EBX, [EBP+EBX]           | ；默认段寄存器SS     |
| MOV EBX, [EAX+100H]          | ；默认段寄存器DS     |
| MOV EDX, ES:[EAX*4+200H]     | ；显式段寄存器ES     |
| MOV [ESP+EDX*2], AX          | ；默认段寄存器SS     |
| MOV EBX, GS:[EAX+EDX*2+300H] | ；显式段寄存器GS     |
| MOV AX, [ESP]                | ；默认段寄存器SS     |

$$EA = \left\{ \begin{array}{c} \text{无} \\ \text{EAX} \\ \text{EBX} \\ \text{ECX} \\ \text{EDX} \\ \text{ESI} \\ \text{EDI} \\ \text{EBP} \\ \text{ESP} \end{array} \right\} + \left\{ \begin{array}{c} \text{无} \\ \text{EAX} \\ \text{EBX} \\ \text{ECX} \\ \text{EDX} \\ \text{ESI} \\ \text{EDI} \\ \text{EBP} \end{array} \right\} \times \left\{ \begin{array}{c} 1 \\ 2 \\ 4 \\ 8 \end{array} \right\} + \left\{ \begin{array}{c} \text{无} \\ 8 \text{ 位} \\ 32 \text{ 位} \end{array} \right\}$$

基址寄存器
变址寄存器
比例因子
偏移常量

# 32位地址的寻址方式

- 在32位微机系统中，除了支持前面的七种寻址方式外，又提供了一种更灵活、方便，但也更复杂的内存寻址方式，从而使内存地址的寻址范围得到了进一步扩大。

在用16位寄存器来访问存储单元时，只能使用基址寄存器(BX和BP)和变址寄存器(SI和DI)来作为地址偏移量的一部分，但在用32位寄存器寻址时，不存在上述限制，所有32位寄存器(EAX、EBX、ECX、EDX、ESI、EDI、EBP和ESP)都可以是地址偏移量的一个组成部分。

当用32位地址偏移量进行寻址时，内存地址的偏移量可分为三部分：一个32位基址寄存器，一个可乘1、2、4或8的32位变址寄存器，一个8位/32位的偏移常量，并且这三部分还可进行任意组合，省去其中之一或之二。

# 32位地址的寻址方式

- 32位基址寄存器是：**EAX、EBX、ECX、EDX、ESI、EDI、EBP和ESP**；  
32位变址寄存器是：**EAX、EBX、ECX、EDX、ESI、EDI和EBP(除ESP之外)**。

下面列举几个32位地址寻址指令：

- `MOV AX, [123456H]`      `MOV EAX, [EBX]`  
`MOV EBX, [ECX*2]`      `MOV EBX, [EAX+100H]`  
`MOV EDX, [EAX*4+200H]`  
`MOV EBX, [EAX+EDX*2]`  
`MOV EBX, [EAX+EDX*2+300H]`  
`MOV AX, [ESP]`

# 8086 CPU指令系统

- 8086 CPU的指令系统是一个数目庞大、形式多样、功能强大的指令系统，它可分为**数据传送类指令、算术运算类指令、逻辑运算与移位类指令、串操作类指令、控制转移类指令、处理器控制指令**等六大类。
- **数据传送类指令**
  - 通用数据传送指令：MOV
  - 交换指令：XCHG
  - 堆栈操作指令：PUSH，POP
  - 地址传送操作指令：LEA，LDS，LES
  - 标志寄存器传送指令：LAHF，SAHF，PUSHF，POPF
  - 累加器专用传送指令：IN，OUT，XLAT

# MOV指令（MOV DST, SRC）

---

- 使用MOV指令要注意以下几点：
  - （1）立即数只能作为源操作数，不能作为目的操作数。
  - （2）立即数不能直接传送到段寄存器，但可通过其他寄存器或堆栈传送。
  - （3）MOV指令的两个操作数类型必须相同。  
例如：MOV AL,BX 是错误的指令。
  - （4）CPU中的寄存器除IP外都可通过MOV指令访问。
  - （5）CS只能作为源操作数，不能作为目的操作数。
  - （6）段寄存器之间不能直接传送，两个内存单元之间不能直接传送。例如：MOV [BX],[SI] 是错误的指令。
  - （7）不影响标志位。

## • 通用数据传送指令

传送指令: **MOV DST, SRC**

执行操作: **(DST)  $\leftarrow$  (SRC)**

注意:

\* **DST、SRC** 不能同时为段寄存器      **MOV DS, ES** ×

\* 立即数不能直接送段寄存器      **MOV DS, 2000H** ×

\* **DST** 不能是立即数和**CS**

\* **DST、SRC** 不能同时为存储器寻址

\* 不影响标志位

**MOV AX, DSEG**

**MOV DS, AX**

# 交换指令XCHG (eXCHanGe)

- 交换指令一般形式为:
- `XCHG reg, mem/reg ;`       $(opr1) \longleftrightarrow (opr2)$
- 不影响标志位
- 该指令可以实现字节交换，也可以实现字交换，可以实现数据在CPU的内部寄存器之间交换，也可以实现数据在CPU内部寄存器和存储单元之间进行交换。例如：
- `XCHG AL, BL;`      AL和BL的内容进行交换
- `XCHG AX, DX;`      AX和DX之间进行字交换
- `XCHG CX, [3450H];`      CX中的内容和3450H、3451H两单元的内容进行交换

# 堆栈操作指令 PUSH, POP

- 在子程序调用时要保存返回地址，在中断处理过程中要保存断点地址，进入子程序和中断处理后还要保留通用寄存器的值。子程序执行完毕和中断处理完毕返回时，又要恢复通用寄存器的值，并分别将返回地址或断点地址恢复到指令指针寄存器中。这些功能都要通过堆栈来实现。8086的堆栈操作必须遵循以下原则：
  - (1) 堆栈的存取每次必须是一个字（16位），而不能是单独一个字节。
  - (2) 堆栈指令中的操作数只能是存储器或寄存器操作数，而不能是立即数。
  - (3) 堆栈指针SP总是指向栈顶。**所谓栈顶，是当前可用堆栈操作指令进行数据交换的存储单元。**
  - (4) 入栈时“**先减后压**”（**SP先减2，再压入操作数**），出栈时“**先弹后加**”（**弹出操作数后，SP加2**）。

# 地址传送指令

## (1) 取有效地址指令LEA (Load **Effective Address**)

- LEA指令的功能是将存储器地址送到一个寄存器（16位）中，指令的源操作数必须是存储器操作数，目的操作数必须是寄存器操作数。
- LEA BP, [3456H] ; 将3456单元的偏移量送BP，执行指令后BP=3456H
- LEA BX, DATA ; 将内存单元DATA的偏移地址送BX，这里DATA为符号地址

## (2) 地址指针装到DS和指定的寄存器指令LDS (Load pointer with DS)

- LDS指令的功能是把2个字的**地址指针**（一个段地址和一个偏移地址）传送到DS和另一个指定的寄存器中，其中地址指针的后一个字（即段地址）送到DS。

# 地址传送指令

---

(3) 地址指针装到ES和指定的寄存器指令LES (Load pointer with ES)

- LES指令格式为: LES reg, mem
- LES指令与LDS指令功能类似, 只是把DS换成ES。操作时将段地址 (后一个字) 传送ES段寄存器。
- **要注意LDS、LES两条指令都是传送一个目的地址指针 (包括一个偏移地址和一个段地址), 共32位数据; 源操作数必须是存储器操作数。**

例: **LEA BX, [BX+SI+0F62H]**  
**LDS SI, [10H]**  
**LES DI, [BX]**

|                                   |             |                             |                     |
|-----------------------------------|-------------|-----------------------------|---------------------|
| <b>TABLE</b><br><b>(DS):1000H</b> |             | <b>MOV BX, TABLE</b>        | <b>; (BX)=0040H</b> |
|                                   | <b>40 H</b> | <b>MOV BX, OFFSET TABLE</b> | <b>; (BX)=1000H</b> |
|                                   | <b>00 H</b> | <b>LEA BX, TABLE</b>        | <b>; (BX)=1000H</b> |
|                                   | <b>00 H</b> | <b>LDS BX, TABLE</b>        | <b>; (BX)=0040H</b> |
|                                   | <b>30 H</b> |                             | <b>; (DS)=3000H</b> |
|                                   |             | <b>LES BX, TABLE</b>        | <b>; (BX)=0040H</b> |
|                                   |             |                             | <b>; (ES)=3000H</b> |

**注意:**

- \* 不影响标志位
- \* **REG** 不能是段寄存器
- \* **SRC** 必须为存储器寻址方式

# 标志寄存器传送指令

---

- (1) 读取标志指令LAHF (Load AH from Flags)
- (2) 设置标志指令SAHF (Store AH into Flags)
- (3) 标志入栈指令 PUSHF (PUSH Flags)
- (4) 标志出栈指令POPF (POP Flags)
- POPF指令与PUSHF指令刚好相反，执行时从堆栈栈顶弹出一个字送到标志寄存器，然后堆栈指针SP的值加2。
- PUSHF指令和POPF指令分别起保护标志和恢复标志的作用。PUSHF，LAHF对标志位无影响，而SAHF、POPF将对标志位产生影响，使当前的标志位被新的值所替换。

# 累加器专用传送指令

- 累加器专用传送指令共有三条，它包括**输入/输出（I/O）指令和换码指令XLAT**。
- 输入/输出指令共有两条：IN和OUT，它用来实现累加器（AX/AL）与I/O端口之间的数据传送，执行输入指令时CPU可以从一个端口（8位）读入一个字节到AL中，也可以从两个连续的8位端口读一个字到AX中。执行输出指令时，CPU可将AL中的一个字节写到一个8位端口中，也可以将AX中的一个字写到两个连续的8位端口中。

8086系统的I/O指令中有以下两种寻址方式：

- 直接寻址方式：**指令中直接指出一个8位的I/O端口地址，端口地址为00~FFH**。
- 寄存器间接寻址方式：**当端口地址大于FFH（100H~FFFFH）时，端口地址由DX寄存器指定**（也只能由DX指定）。当然，DX中也可存放8位的端口地址。

# 累加器专用传送指令

---

## (1) 输入指令IN

- IN指令的一般形式为：
- IN AL/AX, port ; 直接寻址, port为8位立即数表示的端口地址
- 或IN AL/AX, DX ; 间接寻址, 由DX指出8位或16位端口地址

## (2) 输出指令OUT

- OUT指令的一般形式为：
- OUT port, AL/AX ; 直接寻址, port为8位立即数表示的端口地址
- 或OUT DX, AL/AX ; 间接寻址, DX给出8位或16位端口地址

# 累加器专用传送指令

## (3) 换码指令XLAT

- 该指令通过AL和BX寄存器进行表格查找，将8位数装入AL中。它完成的操作为： $AL \leftarrow [BX + AL]$
- XLAT指令常用于查表操作，即BX寄存器含有表格的起始地址，而AL中的值是作进入表格中的偏移量，查出表格中的内容送入AL中。

例如：设DS = 3000H， AL = 09H， BX = 0080H

- 执行指令XLAT作用是：将存储单元30089H的内容送入AL中。

# 判断下列指令书写是否正确

- |                    |                              |
|--------------------|------------------------------|
| (1) MOV AL, BX     | (1)不正确,AL与BX数据宽度不同。          |
| (2) MOV AL, CL     | (2)正确。                       |
| (3) INC [BX]       | (3)不正确,因为不明确是增量字节还是字。        |
| (4) MOV 5, AL      | (4)不正确,立即数不能作为目的操作数。         |
| (5) MOV [BX], [SI] | (5)不正确,不明确要传送的是字节还是字。        |
| (6) MOV BL, 0F5H   | (6)正确。                       |
| (7) MOV DX, 2000H  | (7)正确。                       |
| (8) <b>POP CS</b>  | (8)不正确,CS为代码段的寄存器,不能用pop来改变。 |
| (9) PUSH CS        | (9)正确,可以通过PUSH指令将CS压栈。       |

# 算术运算指令

---

- 8086 CPU指令系统中，具有完备的加、减、乘、除等算术运算指令，有很强的运算能力，它们分别是：
  - 加法指令：ADD, ADC, INC
  - 减法指令：SUB, SBB, DEC, NEG, CMP
  - 乘法指令：MUL, IMUL
  - 除法指令：DIV, IDIV, CBW, CWD
  - 十进制调整指令：AAA, DAA, AAS, DAS, AAM, AAD

# 乘法指令

---

## 无符号乘法指令MUL (MULtiplication)

- 指令格式: MUL mem/reg
- 其作用是将寄存器或存储单元的内容乘以AL或AX的值, 注意AL或AX为隐含操作数。当mem/reg为一个字节时, 另一个乘数为AL的值, 运算的结果(16位)送AX; 若mem/reg为一个字时, 另一个乘数应为AX的值, 运算结果(32位)的**低位字放AX中, 高位字放DX中**。
- 该指令对标志的影响为: 当乘积的高半部分(字节相乘时为AH, 字相乘时为DX)不为零, 则CF=1, OF=1, 代表AH或DX中存放了乘积的有效数字; 否则CF=OF=0。

# 除法指令

---

- 除法指令也分无符号数的除法DIV和带符号数的除法IDIV指令，被除数隐含在累加器AX（字节除）或DX和AX（字除）中。在除法运算中如果除数是8位的，则要求被除数是16位；如果除数是16位的，则要求被除数是32位。

## （1）无符号数的除法指令DIV

- 指令格式：DIV mem/reg
- 指令功能：将AX（16位）中或DX和AX（32位）中的内容，除以在指定的寄存器或存储单元中的内容，对于字节除法，所得的商存于AL，余数存于AH；对于字除法，所得的商存于AX，余数存于DX。

# 逻辑运算和移位指令

---

## 一、逻辑运算类指令

- (1) 逻辑与指令AND
- (2) 逻辑或指令OR
- (3) 逻辑非指令NOT
- (4) 逻辑异或指令XOR
- (5) 测试指令TEST

## 二、移位指令

- (1) 逻辑左移指令SHL (SHift logical Left)
- (2) 逻辑右移指令SHR (SHift logical Right)
- (3) 算术左移指令SAL(Shift Arithmetic Left)
- (4) 算术右移指令SAR(Shift Arithmetic Right)

# 算数左移指令的应用

将AL中的数乘以10的程序段如下：

- `SAL AL, 1` ; 将AL中的内容左移1位，得 $2*AL$
- `MOV BL, AL` ;  $2*AL$ 保存在BL中
- `MOV CL, 02H`
- `SAL AL, CL` ;  $2*AL$ 左移2位，得 $8*AL$
- `ADD AL, BL` ;  $8*AL$ 加上 $2*AL$ ，得 $10*AL$
- 从指令的数量上看，此方法显得繁琐了些，但执行时间却比用乘法指令快得多。

- 逻辑左移=算数左移，右边统一添0
- 逻辑右移，左边统一添0
- 算数右移，左边添加的数和符号有关

# 串操作类指令

- 8086串操作指令有以下特点：
  - 1) 串操作指令都用SI对源操作数进行间接寻址，并假定是在DS段中，而用DI对目的操作数进行间接寻址，并假定是在ES段中（即目的串的地址和源串地址分别由ES: DI、DS: SI提供）。在使用串操作指令之前应先设置好SI和DI的初值。
  - 2) 在同一个段内实现字符串传送时，应将数据段基址和附加段基址设置成同一数值，即DS=ES，此时，仍由SI和DI分别指出源操作数和目的操作数的有效地址。
  - 3) 串操作指令前可加重重复前缀（如REP），通过它们来控制串操作指令的重复执行。
  - 4) 串操作具有方向性，地址的修改与方向标志DF有关：DF=0时，SI、DI作自动增量修改；DF=1时，SI和DI作自动减量修改。因此在执行串操作之前，往往要先设置DF的值（利用指令STD将DF置1，CLD将DF清0）。
  - 5) **串操作指令是唯一一组源操作数和目的操作数都在存储单元的指令。**
  - 6) 串操作指令中有许多参数是隐含约定的。

# 控制转移指令

- 若同时改变CS和IP的转移称为**段间转移**，其目标属性用FAR表示；仅改变IP的转移称为**段内转移**，其目标属性用NEAR表示，对于很短距离的段内转移（-128 ~ +127），可称为**短转移**，用SHORT表示其目标属性。
- **直接转移**：在转移指令中直接指明目标地址的转移称为直接转移。
- **间接转移**：如果转移地址存在某一寄存器或内存单元中，则称为间接转移。
- 若转移地址存放在寄存器中，则只能实现段内间接转移（因为**寄存器间接寻址的最大范围为64K**）；若转移地址存放在内存单元中，则既可实现段内间接转移，也可实现段间间接转移。

# 控制转移指令

---

- 相对转移：目标地址是IP值加上一个偏移量的转移称相对转移。
- 绝对转移：以一个新的值完全代替当前的IP值（CS值可能也发生改变）的转移称为绝对转移。在8086指令系统中，段内直接转移都是相对转移，段内间接转移以及段间转移都是绝对转移。
- 8086提供了四种控制转移指令：无条件转移指令，条件转移指令，循环控制指令和中断指令。**除中断指令外，其它转移类指令都不影响状态标志。**

# 控制转移指令

控制转移指令通过改变CS:IP来控制程序的执行流程。这类指令包括无条件转移指令、条件转移指令、循环指令、子程序调用和返回指令以及中断和中断返回指令。

## (1) 无条件转移

**JMP 跳转**

## (2) 条件转移

|                 |             |
|-----------------|-------------|
| <b>JZ/JNZ</b>   | 结果为零/不为零则转移 |
| <b>JS/JNS</b>   | 结果为负/为正则转移  |
| <b>JO/JNO</b>   | 溢出/不溢出则转移   |
| <b>JP/JNP</b>   | 奇偶位为1/为0则转移 |
| <b>JB/JNB</b>   | 低于/不低于则转移   |
| <b>JBE/JNBE</b> | 低于等于/高于则转移  |
| <b>JL/JNL</b>   | 小于/不小于则转移   |
| <b>JLE/JNLE</b> | 小于等于/大于则转移  |
| <b>JCXZ</b>     | CX为零则转移     |

# 控制转移指令

控制转移指令通过改变CS:IP来控制程序的执行流程。这类指令包括无条件转移指令、条件转移指令、循环指令、子程序调用和返回指令以及中断和中断返回指令。

## (3) 循环指令

|                      |           |
|----------------------|-----------|
| <b>LOOP</b>          | 循环        |
| <b>LOOPZ/LOOPE</b>   | 为零/相等时循环  |
| <b>LOOPNZ/LOOPNE</b> | 不为零/不等时循环 |

## (4) 子程序调用与返回

|             |    |
|-------------|----|
| <b>CALL</b> | 调用 |
| <b>RET</b>  | 返回 |

## (5) 中断及中断返回

|             |       |
|-------------|-------|
| <b>INT</b>  | 中断    |
| <b>INTO</b> | 溢出则中断 |
| <b>IRET</b> | 中断返回  |

# 处理器控制指令

## 一、状态标志位操作指令

清除进位指令 **CLC** (CLear, Carry) ; 该指令使进位标志 CF=0

置1进位位指令 **STC** (SeT Carry) ; 该指令使 CF=1

取反进位位指令 **CMC** (COMplement Carry) ; 该指令使 CF 的值取反。

清除方向标志位指令 **CLD** (CLear Direction) ; 该指令使方向标志 DF=0。

置1方向标志位指令 **STD** (SeT Direction) ; 该指令使 DF=1。

清除中断标志位指令 **CLI** (CLear Interrupt) ; 该指令使中断标志 IF=0

置1中断标志位指令 **STI** (SeT Interrupt) ; 该指令使 IF=1。

# 汇编程序的结构

```
*****  
;  
DATA    SEGMENT                      ; 数据段定义开始  
  
HEX     DB 5AH  
  
ASC     DB 2 DUP (?)  
  
DATA    ENDS                        ; 数据段定义结束  
  
*****  
;  
STACK   SEGMENT                      ; 栈段定义开始  
  
        DB 256 DUP (S)  
  
TOP     EQU $-STACK  
  
STACK   ENDS                        ; 栈段定义结束  
  
*****  
;  
CODE    SEGMENT                      ; 代码段定义开始  
  
        ASSUME CS: CODE, DS: DATA, SS: STACK
```

# 汇编程序的结构

1. Segment 定义
2. Code代码

Code程序:

- 1、简单程序
- 2、分支程序
- 3、循环程序
- 4、子程序

```
*****
;
START: MOV AX, DATA                      ; 主程序开始
        MOV DS, AX
        MOV AX, STACK
        MOV SS, AX
        MOV SP, TOP
        MOV BX, OFFSET ASC
        MOV AL, HEX
        MOV AH, AL
        MOV CL, 4
        SHR AL, CL
        CALL NEAR PTR CONVERT
        MOV [BX], AL
        INC BX
        MOV AL, AH
        CALL NEAR PTR CONVERT
        MOV [BX], AL

HLT                      ; 主程序结束
*****
;
CONVERT PROC                ; 过程(子程序)定义开始
        AND AL, 0FH
        CMP AL, 10
        JB NUM
        ADD AL, 7
NUM:    ADD AL, '0'
        RET
CONVERT ENDP                ; 过程(子程序)定义结束
*****
;
CODE    ENDS                ; 代码段定义结束
        END    START        ; 程序结束
```

# 数据项

- **常数**

必须是固定值，没有属性，是确定的数据。用二进制表示以字母B结尾，八进制用字母Q或者O结尾，十进制用字母D或者省略，也可以用科学表示法，例如735E-2。用十六进制表示时，以字母H结尾，字母A~E开头的必须前面加零，例如0A7F3H。

用字符串表示时，用单引号 ‘ ’ 括起来，例如： ‘ABC’

- **变量**

指存放在存储单元中的值，在程序运行中是可以修改的。所有的变量都具有三个属性。

(1) 段值 (SEGMENT)，即变量的段基址

(2) 段内偏移地址 (OFFSET)，指变量地址与所在段首地址之间的地址偏移字节数

(3) 类型 (TYPE)，变量中每个元素所包含的字节数

# 数据项

---

- 标号

**可执行指令语句的地址的符号表示**，它可作为转移指令的目标操作数，以确定程序转向的目标地址，它具有三个属性：

- （1）段值（SEGMENT），即标号的段基址
- （2）段内偏移地址（OFFSET），指标号地址与所在段首地址之间的地址偏移字节数
- （3）类型（TYPE），标号的类型属性指在转移指令中标号可转移的距离，也称为距离属性。近标号NEAR，段内转移或调用，远标号FAR，段间转移或调用。

# MASM中的表达式

|       |                                             |                                                        |                                                         |                                                                                                                                                                                                                                                          |
|-------|---------------------------------------------|--------------------------------------------------------|---------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 算术运算符 | +<br>-<br>*<br>/<br>MOD                     | 加法<br>减法<br>乘法<br>除法<br>模除                             | 和<br>差<br>乘积<br>商<br>余数                                 | $3+5=8$<br>$8-3=5$<br>$3*5=15$<br>$21/7=3$<br>$12\text{MOD}3=0$                                                                                                                                                                                          |
| 逻辑运算符 | NOT<br>AND<br>OR<br>XOR                     | “非”运算<br>“与”运算<br>“或”运算<br>“异或”运算                      | 逻辑“非”结果<br>逻辑“与”结果<br>逻辑“或”结果<br>逻辑“异或”结果               | $\text{NOT } 1010\text{B}=0101\text{B}$<br>$1010\text{B AND } 1001\text{B}=1000\text{B}$<br>$1010\text{B OR } 1001\text{B}=1011\text{B}$<br>$1010\text{B XOR } 1001\text{B}=0011\text{B}$                                                                |
| 关系运算符 | EQ<br>NE<br>LT<br>LE<br>GT<br>GE            | 相等<br>不等<br>小于<br>不大于（小于等于）<br>大于<br>不小于（大于等于）         | 结果为真输出全“1”<br>结果为假输出全“0”                                | $5 \text{ EQ } 11\text{B}=\text{全“0”}$<br>$5 \text{ NE } 11\text{B}=\text{全“1”}$<br>$5 \text{ LT } 11\text{B}=\text{全“0”}$<br>$5 \text{ LE } 11\text{B}=\text{全“0”}$<br>$5 \text{ GT } 11\text{B}=\text{全“1”}$<br>$5 \text{ GE } 11\text{B}=\text{全“1”}$ |
| 分析运算符 | SEG<br>OFFSET<br>LENGTH<br>TYPE<br>SIZE     | 返回段基址<br>返回偏移地址<br>返回变量单元数<br>返回元素字节数<br>返回变量总字节数      | 段基址<br>偏移地址<br>单元数<br>字节数<br>总字节数                       | $\text{SEG N1}=\text{N1 所在段段基址}$<br>$\text{OFFSET N1}=\text{N1 的偏移地址}$<br>$\text{LENGTH N1}=\text{N1 单元数}$<br>$\text{TYPE N1}=\text{N1 中元素字节数}$<br>$\text{SIZE N1}=\text{N1 总字节数}$                                                                       |
| 合成运算符 | PTR<br>THIS<br>段操作码<br>HIGH<br>LOW<br>SHORT | 修改类型属性<br>指定类型/距离属性<br>段前缀<br>分离高字节<br>分离低字节<br>短程转移说明 | 修改后类型<br>指定后类型<br>修改段<br>运算对象的高字节<br>运算对象的低字节<br>实现短程转移 | $\text{BYTE PTR[BX]}$<br>$\text{ALP EQU THIS BYTE}$<br>$\text{ES:[BX]}$<br>$\text{HIGH } 3050\text{H}=30\text{H}$<br>$\text{LOW } 3050\text{H}=50\text{H}$<br>$\text{JMP SHORT LOOP1}$                                                                   |

# 算术运算符（arithmetic operator）

- 算术运算符包括+（加）、-（减）、\*（乘）、/（除，只取商）和MOD（模除，只取余数）五种运算，算术运算符通常用于数字表达式或地址表达式中。在将它们用于地址表达式中时，一般应采用在标号上加减某一个数字量，如START+3, SUM-1等。用这样的表达式来表示一个存储单元的地址。

- 给出下面程序段：

```
A1 EQU 1020H+3300H
```

```
MOV BX, A1-1000H
```

```
MOV AX, 35*5
```

```
MOV DX, A1/100H
```

```
MOV CX, A1 MOD 100H
```

则可等效为：

```
A1 EQU 4320H
```

```
MOV BX, 3320H
```

```
MOV AX, 175
```

```
MOV DX, 0043H
```

```
MOV CX, 0020H
```

# 逻辑运算符（logical operator）

- 包括NOT（非）、AND（与）、OR（或）和XOR（异或）四种。这些逻辑运算符完成的运算是按位操作的，只能用于数字表达式中，不能用于存储器的地址表达式中，如：
- NOT 0FFH 结果为00H
- 11001100B AND 11110000B 结果为1100 0000B
- 11001100B OR 11110000B 结果为1111 1100B
- 11001100B XOR 11110000B 结果为0011 1100B

注意：这些逻辑运算符与指令系统中的逻辑运算指令助记符在形式上是相同的，但两者有显著的区别。**表达式中的逻辑运算符由汇编程序来完成运算，而逻辑运算指令要在CPU执行该指令时才完成相应的操作。**

# 关系运算符（relational operator）

- 包括EQ（相等）、NE（不等）、LT（小于）、LE（不大于/小于或等于）、GT（大于）、GE（不小于/大于或等于）六种运算。
- 关系运算符连接的两个操作数必须都是数字的或是同一段内的两个存储器地址。关系运算符对两个运算对象进行比较操作，运算的结果是逻辑值，若满足条件，则表示运算结果为真（TRUE），输出结果全为1；若不满足条件，则表示运算结果为假（FALSE），这时输出结果为全0。
- |                    |   |                |
|--------------------|---|----------------|
| MOV AX, 5 EQ 101B  | → | MOV AX, 0FFFFH |
| MOV BH, 10H GT 16  | → | MOV BF, 00H    |
| ADD BL, FFH EQ 255 | → | ADD BL, 0FFH   |

# 分析运算符（analytic operator）

- 又称数值返回运算符，包括SEG, OFFSET, LENGTH, TYPE和SIZE 五种。分析运算符总是加在运算对象之前，返回的结果是运算对象的某个参数或将存储器地址分解为它的组成部分，如段基址、偏移地址和类型等。
- SEG运算符加于某个变量或标号之前，返回的数值是该变量或标号的段基址。
- OFFSET运算符加于某个变量或标号之前，返回的数值是该变量或标号的偏移地址。
- LENGTH运算符加于某个变量之前，返回的数值是该变量所包含的单元数，分配单元可以以字节、字、双字为单位计算。对于变量中使用DUP的情况，汇编程序将返回分配给变量的单元数，而对于其他情况则返回“1”值。
- SIZE运算符加于某个变量之前，返回的数值是该变量所包含的总字节数，此值是该变量LENGTH值和TYPE值的乘积。

# 合成运算符（synthetic operator）

---

- 又称为修改属性运算符。在程序运行过程中，当需要修改变量或标号的属性（段属性、偏移地址属性和类型属性）时，可采用合成运算符来实现。
- **PTR运算符可用来修改变量或标号的类型属性**。类型可以是BYTE, WORD, DWORD, NEAR和FAR，表达式可以是变量、标号或存储器操作数，其含义是将PTR左边的类型属性赋给其右边的表达式。
- THIS为指定类型属性运算符，可用来定义变量或标号的类型属性。THIS运算符的对象是类型（BYTE, WORD, DWORD）或距离（NEAR, FAR），用于规定所指变量或标号的类型属性或距离属性，时常和EQU伪指令连用。

# 合成运算符（synthetic operator）

---

- 段操作码 也称为段超越前缀。用来表示一个标号、变量或地址表达式的段属性。例如，用段超越前缀来说明地址是在附加段中，可用指令MOV DX, ES: [BX][DI]，可见它是用“段寄存器名：地址表达式”来表示的。
- 分离运算符有HIGH和LOW两种。HIGH运算符用来从运算对象中分离出高字节，而LOW运算符用来从运算对象中分离出低字节。
- SHORT为说明运算符，它说明转移指令的目标地址与本指令之间的字节距离在-128~+127范围内，具有短程转移的属性。

# 运算符的优先级规定

| 优 先 级                                    |    | 符 号                                      |
|------------------------------------------|----|------------------------------------------|
| <div>高级</div> <div>↑</div> <div>低级</div> | 1  | LENGTH, SIZE, WIDTH, MASK, ( ), [ ], < > |
|                                          | 2  | PTR, OFFSET, SEG, TYPE, THIS             |
|                                          | 3  | HIGH, LOW                                |
|                                          | 4  | +, - (单目)                                |
|                                          | 5  | *, /, MOD                                |
|                                          | 6  | +, - (双目)                                |
|                                          | 7  | EQ, NE, LT, LE, GT, GE                   |
|                                          | 8  | NOT                                      |
|                                          | 9  | AND                                      |
|                                          | 10 | OR, XOR                                  |
|                                          | 11 | SHORT                                    |

# 变量定义伪指令

- 变量定义伪指令用来定义变量的类型，并为变量中的数据项分配存储单元。变量定义伪指令有两种不同的格式。

- **1. 格式1 [变量名] DB/DW/DD/DQ/DT 表达式**

功能如下：

DB (define byte)：定义一个字节类型的变量，其后的每个操作数均占用1个字节。

DW (define word)：定义一个字类型的变量，其后的每个操作数均占用1个字（2个字节）。

DD (define doubleword)：定义一个双字类型的变量，其后的每个操作数均占用2个字（4个字节）。

DQ (define quadword)：定义一个四字类型的变量，其后的每个操作数均占用4个字（8个字节）。

DT (define ten bytes)：定义一个十字节类型的变量，其后的每个操作数均占用5个字（10个字节）。

# 变量定义伪指令

- 格式2用于定义重复变量，其格式如下所示。

$$\text{变量名} \left\{ \begin{array}{l} \text{DB} \\ \text{DW} \\ \text{DD} \\ \text{DQ} \\ \text{DT} \end{array} \right. \quad n \quad \text{DUP} \quad (\text{操作数})$$

与格式1的不同之处在于格式2增加了  $n$  DUP (duplicate) 用于表示重复次数，同时表达式需用圆括号括起。其中，重复次数  $n$  可以是常数，也可以是表达式，它的值应该是一个正整数，数值范围为1~65535，其作用是指定括号中操作数项的重复次数。括号中的操作数项可以有多项，但项与项之间也必须用逗号分隔开来。这种格式适用于定义许多相同的变量。

# 符号定义伪指令

- 汇编语言中所有变量名、标号名、过程名、指令助记符、寄存器名等统称为“**符号**”。这些符号可以用符号定义伪指令来命名或重新命名。**伪指令不占用内存**，有两种基本格式。
- 等值语句（EQU）

等值语句在汇编时不产生任何目标代码，也不占用存储单元。但在同一个程序中，不能对经EQU语句定义的符号重新定义。EQU伪指令的使用可使汇编语言程序简单明了，便于程序调试和修改。
- 等号语句（=）

等号语句与EQU语句有同样的功能，区别在于等号语句定义的符号允许重新定义，使用更加方便灵活。

# 段定义伪指令

---

- 段定义伪指令指示汇编程序如何按段组织程序和使用存储器。段定义伪指令主要有SEGMENT/ENDS, ASSUME和ORG。在MASM 5.0以上的汇编语言版本中，有完整段定义伪指令和简化段定义伪指令两种。

- 段定义伪指令（SEGMENT/ENDS）

格式： **段名SEGMENT** [定位类型][, 组合类型][, 字长类型][, 类别]

( 段体 ) :

:

**段名 ENDS**

# 段定义伪指令

---

- 段分配伪指令（**ASSUME**）
- 段分配伪指令用来完成段的分配，说明当前哪些逻辑段被分别定义为代码段、数据段、堆栈段和附加段。
- 代码段用来存放被执行的程序；数据段用来存放程序执行中需要的数据和运算结果；当用户程序中使用的数据量很大或使用了串操作指令时，可设置附加段来增加数据段的容量；堆栈段用来设置堆栈。
- 在用ASSUME语句来完成段的分配时，要注意以下几点。
  - ① 在一个代码段中，如果没有另外的ASSUME语句重新设置，则原有的ASSUME语句的设置一直有效。
  - ② 每条ASSUME语句可设置1~6个段寄存器。
  - ③ 可以使用NOTHING将以前的设置删除。
  - ④ 段寄存器的装入。

# 过程定义伪指令

---

- 过程定义伪指令（**PROC/ENDP**）

格式：过程名 PROC 属性

:

RET

过程名 ENDP

其中，过程名是个标识符，写法规定与标号相同，可以将过程名理解为是过程入口的符号地址，调用过程时只要在CALL指令后写上该过程名即可。属性字段用来指明过程的类型属性是NEAR还是FAR。RET指令总是设置在过程体的末尾，使过程结束后可以返回主程序，PROC和ENDP是过程定义的伪指令，PROC为起始语句，ENDP为结束语句，这两条语句之间的内容就是被定义的过程，PROC和ENDP前面的过程名应是相同的，并且PROC和ENDP总是成对出现。

# 其他伪指令

- LABEL伪指令

LABEL伪指令可为要使用的变量或标号定义一个类型。

对于变量，类型可以是BYTE, WORD或DWORD；对于标号，类型可以为NEAR或FAR。

ABC LABEL BYTE ; 将变量ABC定义为字节类型

XYZ LABEL WORD ; 将变量XYZ定义为字类型

- 指定起始位置伪指令（ORG）和当前位置计数器（\$）

ORG伪指令指出其后的程序段或数据块存放的起始地址的偏移量。地址计数器值可用“\$”来表示，在汇编程序对源程序进行汇编的过程中，使用地址计数器来保证当前正在汇编的指令地址。汇编语言也允许用户直接用“\$”来引用地址计数器的当前值，因此，ORG \$+5可表示从当前地址开始跳过5个字节存储单元，在指令和伪指令中，也可直接用“\$”表示地址计数器的当前值。

# 程序开始伪指令

---

- 有两种指令格式，用来指示汇编语言源程序的开始，为模块取名字。

## (1) NAME伪指令

格式：NAME 模块

汇编程序将以给出的模块名作为模块的名字。

## (2) TITLE伪指令

格式：TITLE 文本

TITLE伪指令可指定所打印模块的标题。如程序中没有NAME伪指令，则汇编程序将用文本中的前6个字符作为模块名。文本最多可有60个字符。

如果程序中既无NAME又无TITLE伪指令时，汇编程序将用源文件名作为模块名。所以在编制程序时，NAME和TITLE伪指令并不是必要的。但一般经常使用TITLE伪指令，以便在列表文件中能打印出标题来。

# 汇编语言程序的基本格式

---

- DATA SEGMENT  
; 存放数据项的数据段  
DATA ENDS  
EXTRA SEGMENT  
; 存放数据项的附加段  
EXTRA ENDS  
STACK1 SEGMENT PARA STACK  
; 堆栈段  
STACK1 ENDS  
CODE SEGMENT  
ASSUME CS: CODE, DS: DATA, SS: STACK1, ES: EXTRA  
START: MOV AX, DATA  
MOV DS, AX ; 段基址装入DS  
MOV AX, EXTRA  
MOV ES, AX ; 段基址装入ES  
MOV AX, STACK1  
MOV SS, AX ; 段基址装入SS  
; 核心程序段  
MOV AH, 4CH ; 系统功能调用  
INT 21H ; 返回操作系统  
CODE ENDS  
END START

# 汇编语言程序设计的基本步骤

---

- 与高级语言程序设计类似，用汇编语言进行程序设计同样按以下步骤进行。
- ① 分析问题，建立数学模型。
- ② 确定算法。
- ③ 编制程序流程图。
- ④ 编写程序。
- ⑤ 上机调试